AURA — Project Summary

Agent Uncertainty and Residue Arbiter

Research prototype | October 2026

Overview

AURA is a lightweight framework for independently checking whether a multi-step AI-agent task has truly been completed. An agent's success message is not sufficient evidence that the required state was reached or that no unwanted changes remain.

Problem

Multi-step agents can encounter silent tool failures, partial updates, inconsistent records, or unintended side effects. Individual tool responses may look successful while the overall task remains incomplete. AURA evaluates observable outcomes against explicit task requirements.

What we will build

1. **Workflow simulator (Python):** Runs realistic multi-step operations involving records and files.

2. **SQLite state store:** Holds the state that workflows change and the verifier inspects.

3. **Task contracts:** Specify required final conditions, permitted changes, and forbidden side effects.

4. **Failure injector:** Reproducibly introduces failed, partial, or misleading operations.

5. **Independent AURA verifier:** Performs targeted inspections and checks contract postconditions and residual side effects.

6. **Evidence records:** Logs observations, verification results, and reasons for decisions.

7. **Evaluation scripts:** Calculate failure-detection measures, false alarms, and runtime overhead.

Workflow

Task + contract → workflow simulator → changed system state → AURA inspections → evidence → **PASS / FAIL / UNCERTAIN**.

PASS: Required conditions have sufficient supporting evidence and no forbidden changes are detected.

FAIL: An observed condition violates the contract.

UNCERTAIN: Available evidence cannot establish success or failure confidently.

Experiments

Compare three approaches on the same controlled scenarios:

• **Baseline:** Trust ordinary execution status and reported success.

• **Per-tool checks:** Verify individual tool operations.

• **AURA:** Independently verify the complete workflow's final state and side effects.

Record detection rates, false positives, uncertainty rates, verification cost, and representative failure cases. Do not assume AURA will outperform every baseline; measure it.

Technology and scope

Python, SQLite, JSON, pytest, pandas, and Matplotlib. The first pilot is local and requires neither a GPU nor paid APIs. It is a **controlled agent-workflow simulator**, not a claim of production-ready verification of arbitrary

autonomous agents.

Proposed three-day pilot

Day 1: Implement state store, scenarios, and contracts.

Day 2: Implement failure injection, verification, and evidence logging.

Day 3: Run comparative tests, create charts, document findings and limitations.

Deliverables

Working source code; reproducible test cases; contract examples; failure-injection scripts; evidence logs; CSV results; graphs; README; and a concise research report.

Research objective

Evaluate whether contract-defined, evidence-based workflow verification improves the reliability of multi-step agent task-completion judgments compared with trusting execution status or checking tools separately.

Boundaries and next steps

The initial version does not require an LLM API, cloud infrastructure, or integration with real services. After validating the pilot, optional extensions include real tool-using agents, richer uncertainty policies, and more complex cross-service workflows.